

The Unix Shell + Intro to Git

26 February 2025

09:30 – 14:30

Meron en Stephanie
VU University Library



Setup

If you haven't done so yet, please follow the setup instructions for this lesson and download the data at:

<https://edu.nl/xwhyq>

Download the data to your **Downloads** folder and **unzip** them (give it the suggested name shell-lesson-data)

Outline for today

09:30-12:20 The Unix Shell

12:20-13:00 Lunch break

13:00-14:30 Intro to Git

No worries: breaks in between!



About folders/directories and files



Bundesarchiv, B 145 Bild-P109966 / CC-BY-SA
3.0

Some analogies:
Cabinet → 'drive'/root
Folder → folder
Paper file → digital file

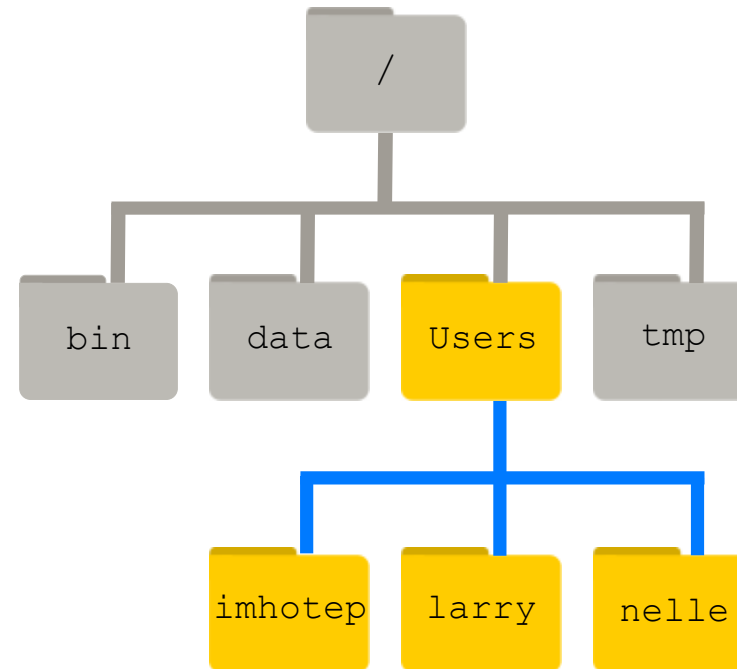
This tells us something
about how the file
system on our
computer is organised

Directories

Paths: /Users/imhotep

/Users/larry

/Users/nelle



Exercise: Absolute vs. Relative Paths

Starting from `/Users/amanda/data`, which of the following commands could Amanda use to navigate to her home directory, which is `/Users/amanda`?

`cd .`

`cd /`

`cd /home/amanda`

`cd ../..`

`cd ~`

`cd home`

`cd ~/data/..`

`cd`

`cd ..`

Exercise: Reading Comprehension

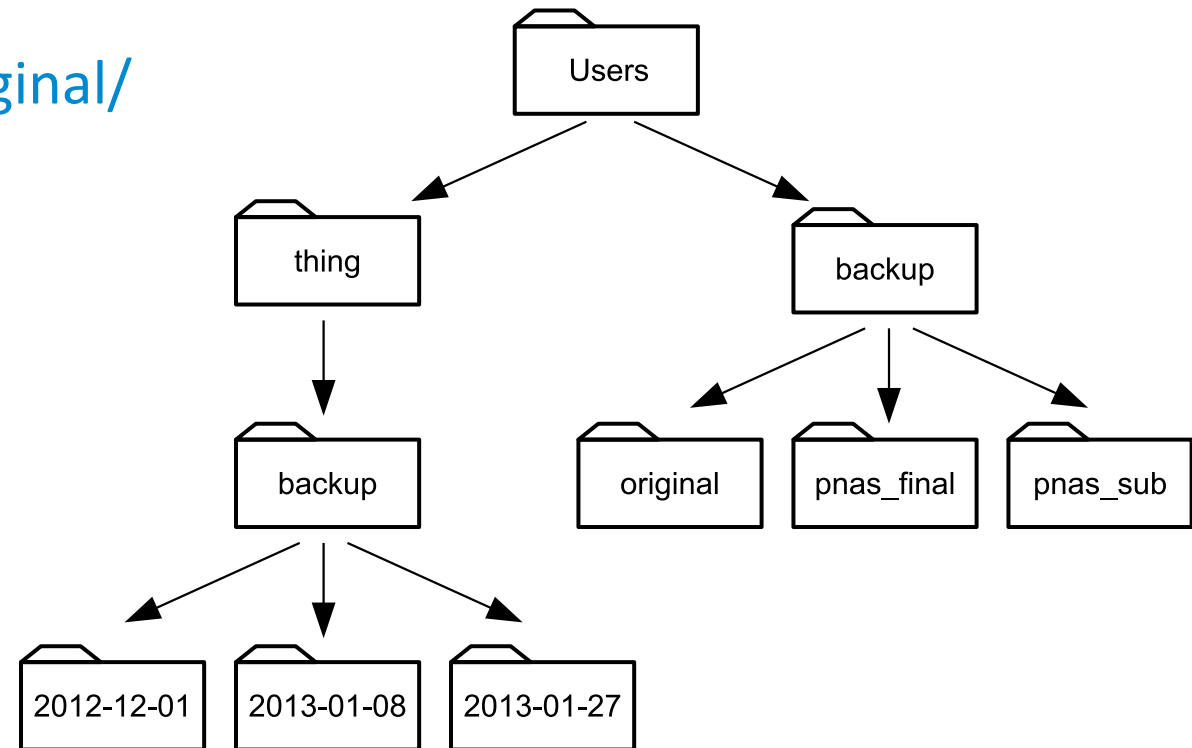
Using the filesystem diagram below, if `pwd` displays `/Users/backup`, and `-r` tells `ls` to display things in reverse order, what command(s) will result in the following output:

■ `pnas_sub/` `pnas_final/` `original/`

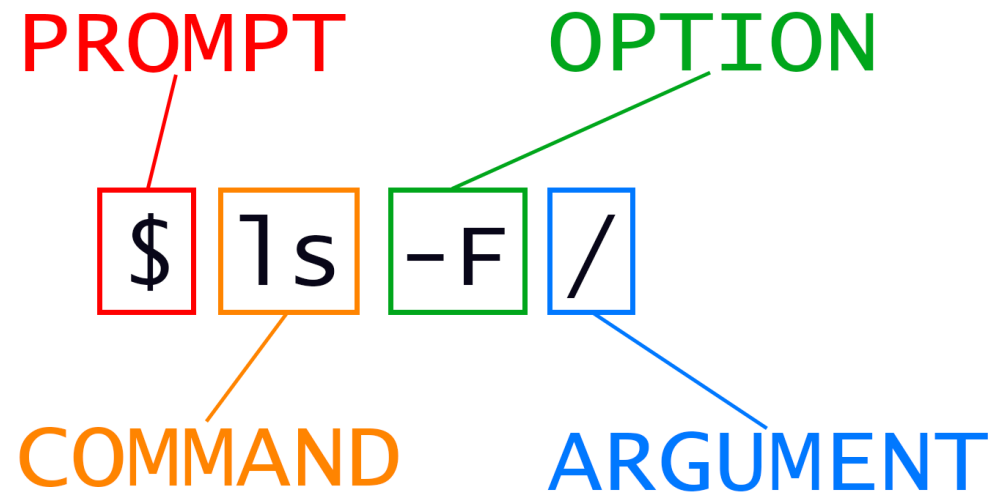
1. `ls pwd`

2. `ls -r -F`

3. `ls -r -F /Users/backup`



General Syntax of a Shell command



Exercise: Renaming files

Suppose that you created a plain-text file in your current directory to contain a list of the statistical tests you will need to do to analyze your data, and named it: `statstics.txt`

After creating and saving this file you realize you misspelled the filename! You want to correct the mistake, which of the following commands could you use to do so?

1. `cp statstics.txt statistics.txt`
2. `mv statstics.txt statistics.txt`
3. `mv statstics.txt .`
4. `cp statstics.txt .`

Exercise: Moving and Copying

What is the output of the closing ls command in the sequence shown below?

```
$ pwd
```

```
/Users/jamie/data
```

```
$ ls
```

```
proteins.dat
```

```
$ mkdir recombined
```

```
$ mv proteins.dat recombined/
```

```
$ cp recombined/proteins.dat ../proteins-saved.dat
```

```
$ ls
```

Answers:

- 1.proteins-saved.dat recombined
- 2.recombined
- 3.proteins.dat recombined
- 4.proteins-saved.dat

Exercise: wildcards

When run in the alkanes directory, which ls command(s) will produce this output?

ethane.pdb methane.pdb

1. ls *t*ane.pdb
2. ls *t?ne.*
3. ls *t??ne.pdb
4. ls ethane.*

Exercise: Organising directories and files

Jamie is working on a project, and she sees that her files aren't very well organised:

```
$ ls -F
```

```
analysed/ fructose.dat  raw/  sucrose.dat
```

The fructose.dat and sucrose.dat files contain output from her data analysis. What command(s) covered in this lesson does she need to run so that the commands below will produce the output shown?

```
$ ls -F
```

```
analysed/  raw/
```

```
$ ls analysed
```

```
fructose.dat  sucrose.dat
```

The sort command

The file `shell-lesson-data/exercise-data/numbers.txt` contains the following lines:

```
10
2
19
22
6
```

If we run `sort` on this file, the output is:

```
10
19
2
22
6
```

If we run `sort -n` on the same file, we get this instead:

```
2
6
10
19
22
```

Explain why `-n` has this effect.

What does >> mean?

We have seen the use of >, but there is a similar operator >> which works slightly differently. We'll learn about the differences between these two operators by printing some strings. We can use the echo command to print strings e.g.

```
$ echo The echo command prints text
```

The echo command prints text

Now test the commands below to reveal the difference between the two operators:

```
$ echo hello > testfile01.txt
```

and:

```
$ echo hello >> testfile02.txt
```

Hint: Try executing each command twice in a row and then examining the output files.

Appending data

We have already met the head command, which prints lines from the start of a file. tail is similar, but prints lines from the end of a file instead.

Consider the file shell-lesson-data/exercise-data/animal-counts/animals.csv. After these commands, select the answer that corresponds to the file animals-subset.csv:

```
$ head -n 3 animals.csv > animals-subset.csv
```

```
$ tail -n 2 animals.csv >> animals-subset.csv
```

1. The first three lines of animals.csv
2. The last two lines of animals.csv
3. The first three lines and the last two lines of animals.csv
4. The second and third lines of animals.csv

Exercise: Piping commands together

In our current directory, we want to find the 3 files which have the least number of lines. Which command listed below would work?

1. `wc -l * > sort -n > head -n 3`
2. `wc -l * | sort -n | head -n 1-3`
3. `wc -l * | head -n 3 | sort -n`
4. `wc -l * | sort -n | head -n 3`

Try this in the `shell-lesson-data/exercise-data/alkanes` directory and try to understand why things work the way they do

Exercise: Pipe Reading Comprehension

A file called `animals.csv` (`shell-lesson-data/exercise-data/animal-counts`) contains the following data:

`2012-11-05,deer,5`

`2012-11-05,rabbit,22`

`2012-11-05,raccoon,7`

`2012-11-06,rabbit,19`

`2012-11-06,deer,2`

`2012-11-06,fox,4`

`2012-11-07,rabbit,16`

`2012-11-07,bear,1`

What text passes through each of the pipes and the final redirect in the pipeline below? Note, the `sort -r` command sorts in reverse order.

```
$ cat animals.csv | head -n 5 | tail -n 3 | sort -r > final.txt
```

Hint: build the pipeline up one command at a time to test your understanding

Optional Exercise: Pipe Construction

For the file `animals.csv` from the previous exercise, consider the following command:

```
$ cut -d , -f 2 animals.csv
```

The `cut` command is used to remove or 'cut out' certain sections of each line in the file, and `cut` expects the lines to be separated into columns by a `Tab` character. A character used in this way is called a **delimiter**. In the example above we use the `-d` option to specify the comma as our delimiter character. We have also used the `-f` option to specify that we want to extract the second field (column). This gives the following output:

```
deer  
rabbit  
raccoon  
rabbit  
deer  
fox  
rabbit  
bear
```

```
$ cut -d , -f 2 animals.csv | sort | uniq
```

The `uniq` command filters out adjacent matching lines in a file. How could you extend this pipeline (using `uniq` and another command) to find out what animals the file contains (without any duplicates in their names)?

Optional Exercise: Which pipe?

The file `animals.csv` contains 8 lines of data formatted as follows:

`2012-11-05,deer,5`

`2012-11-05,rabbit,22`

`2012-11-05,raccoon,7`

`2012-11-06,rabbit,19`

...

The `uniq` command has a `-c` option which gives a count of the number of times a line occurs in its input. Assuming your current directory is `shell-lesson-data/exercise-data/animal-counts`, what command would you use to produce a table that shows the total count of each type of animal in the file?

1. `sort animals.csv | uniq -c`
2. `sort -t, -k2,2 animals.csv | uniq -c`
3. `cut -d, -f 2 animals.csv | uniq -c`
4. `cut -d, -f 2 animals.csv | sort | uniq -c`
5. `cut -d, -f 2 animals.csv | sort | uniq -c | wc -l`